

Redes Neuronales

El Perceptrón Multicapa (MLP)

Sergio Hernández
shernandez@ucm.cl



Universidad Católica del Maule

- ▶ Los modelos lineales (regresión lineal, *softmax*) suponen una relación **afín** entre las entradas y la salida.

¹Contenido basado en Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 5. <https://d2l.ai>

Hacia la primera red *profunda*

- ▶ Los modelos lineales (regresión lineal, *softmax*) suponen una relación **afín** entre las entradas y la salida.
- ▶ Linealidad implica **monotonicidad**: aumentar una variable siempre mueve la predicción en la misma dirección. Esto es a menudo falso.
 - ▶ *Ejemplo*: la probabilidad de impago no crece de forma lineal con el ingreso; los píxeles de una imagen no determinan la categoría de forma afín.

¹Contenido basado en Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 5. <https://d2l.ai>

Hacia la primera red *profunda*

- ▶ Los modelos lineales (regresión lineal, *softmax*) suponen una relación **afín** entre las entradas y la salida.
- ▶ Linealidad implica **monotonicidad**: aumentar una variable siempre mueve la predicción en la misma dirección. Esto es a menudo falso.
 - ▶ *Ejemplo*: la probabilidad de impago no crece de forma lineal con el ingreso; los píxeles de una imagen no determinan la categoría de forma afín.
- ▶ **Idea central**: apilar varias capas de neuronas, cada una *totalmente conectada* a la anterior, e intercalar *no linealidades*.

¹Contenido basado en Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 5. <https://d2l.ai>

Hacia la primera red *profunda*

- ▶ Los modelos lineales (regresión lineal, *softmax*) suponen una relación **afín** entre las entradas y la salida.
- ▶ Linealidad implica **monotonicidad**: aumentar una variable siempre mueve la predicción en la misma dirección. Esto es a menudo falso.
 - ▶ *Ejemplo*: la probabilidad de impago no crece de forma lineal con el ingreso; los píxeles de una imagen no determinan la categoría de forma afín.
- ▶ **Idea central**: apilar varias capas de neuronas, cada una *totalmente conectada* a la anterior, e intercalar *no linealidades*.

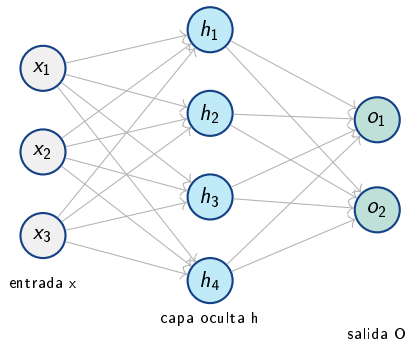
Perceptrón multicapa (MLP)

La red profunda más simple: capas de neuronas donde cada una recibe de la capa inferior e influye en la superior. Con suficientes neuronas y datos, aprende relaciones que un modelo lineal no puede representar.¹

¹Contenido basado en Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 5. <https://d2l.ai>

Arquitectura de un MLP con una capa oculta

- ▶ **Capa de entrada:** no realiza cálculo, sólo distribuye x .
- ▶ **Capa oculta:** neuronas con activación no lineal.
- ▶ **Capa de salida:** produce la predicción O .
- ▶ Es una red *totalmente conectada* (*fully connected*): cada neurona se une a todas las de la capa vecina.



Esta red tiene 3 entradas, 4 unidades ocultas y 2 salidas. Sus parámetros aprendibles son 2 matrices de pesos ($W^{(1)} \in \mathbb{R}^{3 \times 4}$, $W^{(2)} \in \mathbb{R}^{4 \times 2}$) y 2 vectores de sesgo.

Un MLP lineal *colapsa* a un modelo lineal

Con un *minibatch* $X \in \mathbb{R}^{n \times d}$, una capa oculta de H unidades y **sin** función de activación:

$$H = XW^{(1)} + b^{(1)}, \quad O = HW^{(2)} + b^{(2)}.$$

Un MLP lineal *colapsa* a un modelo lineal

Con un *minibatch* $X \in \mathbb{R}^{n \times d}$, una capa oculta de H unidades y **sin** función de activación:

$$H = XW^{(1)} + b^{(1)}, \quad O = HW^{(2)} + b^{(2)}.$$

Sustituyendo H en O :

$$O = (XW^{(1)} + b^{(1)})W^{(2)} + b^{(2)} = X \underbrace{W^{(1)}W^{(2)}}_W + \underbrace{b^{(1)}W^{(2)} + b^{(2)}}_b.$$

Un MLP lineal *colapsa* a un modelo lineal

Con un *minibatch* $X \in \mathbb{R}^{n \times d}$, una capa oculta de H unidades y **sin** función de activación:

$$H = XW^{(1)} + b^{(1)}, \quad O = HW^{(2)} + b^{(2)}.$$

Sustituyendo H en O :

$$O = (XW^{(1)} + b^{(1)})W^{(2)} + b^{(2)} = X \underbrace{W^{(1)}W^{(2)}}_W + \underbrace{b^{(1)}W^{(2)} + b^{(2)}}_b.$$

Conclusión

Componer transformaciones afines produce otra transformación afín: la capa oculta es **inútil** a menos que apliquemos una *no linealidad* $\sigma(\cdot)$ a cada unidad.

Un MLP lineal *colapsa* a un modelo lineal

Con un *minibatch* $X \in \mathbb{R}^{n \times d}$, una capa oculta de H unidades y **sin** función de activación:

$$H = XW^{(1)} + b^{(1)}, \quad O = HW^{(2)} + b^{(2)}.$$

Sustituyendo H en O :

$$O = (XW^{(1)} + b^{(1)})W^{(2)} + b^{(2)} = X \underbrace{W^{(1)}W^{(2)}}_W + \underbrace{b^{(1)}W^{(2)} + b^{(2)}}_b.$$

Conclusión

Componer transformaciones afines produce otra transformación afín: la capa oculta es **inútil** a menos que apliquemos una *no linealidad* $\sigma(\cdot)$ a cada unidad.

$$H = \sigma(XW^{(1)} + b^{(1)}), \quad O = HW^{(2)} + b^{(2)}.$$

- ▶ Apilando capas ocultas con activaciones no lineales obtenemos modelos cada vez más expresivos.

- ▶ Apilando capas ocultas con activaciones no lineales obtenemos modelos cada vez más expresivos.
- ▶ **Teorema de aproximación universal**: un MLP con una sola capa oculta y suficientes neuronas puede aproximar *cualquier* función continua sobre un dominio acotado, con la precisión deseada.

- ▶ Apilando capas ocultas con activaciones no lineales obtenemos modelos cada vez más expresivos.
- ▶ **Teorema de aproximación universal**: un MLP con una sola capa oculta y suficientes neuronas puede aproximar *cualquier* función continua sobre un dominio acotado, con la precisión deseada.
- ▶ Pero “poder representarla” no es lo mismo que “poder aprenderla”:
 - ▶ Una capa muy ancha puede ser inviable de entrenar.
 - ▶ En la práctica, redes más **profundas** (varias capas) suelen ser más eficientes en parámetros que una única capa muy ancha.

- ▶ Apilando capas ocultas con activaciones no lineales obtenemos modelos cada vez más expresivos.
- ▶ **Teorema de aproximación universal**: un MLP con una sola capa oculta y suficientes neuronas puede aproximar *cualquier* función continua sobre un dominio acotado, con la precisión deseada.
- ▶ Pero “poder representarla” no es lo mismo que “poder aprenderla”:
 - ▶ Una capa muy ancha puede ser inviable de entrenar.
 - ▶ En la práctica, redes más **profundas** (varias capas) suelen ser más eficientes en parámetros que una única capa muy ancha.
- ▶ La elección de la no linealidad σ es clave para el entrenamiento.

$$\text{ReLU}(x) = \max(x, 0)$$

- ▶ Es la activación **por defecto**: simple y eficaz.
- ▶ Derivada muy simple:

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

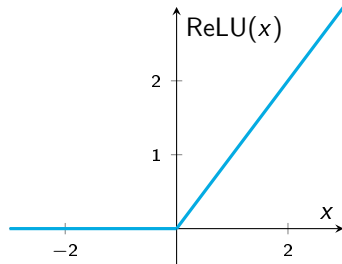
ReLU: unidad lineal rectificada

$$\text{ReLU}(x) = \max(x, 0)$$

- ▶ Es la activación **por defecto**: simple y eficaz.
- ▶ Derivada muy simple:

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

- ▶ Mitiga el *desvanecimiento del gradiente*: para $x > 0$ el gradiente pasa intacto.
- ▶ Variante *pReLU*: $\max(x, 0) + \alpha \min(x, 0)$ deja pasar algo de información cuando $x < 0$.



Sigmoide: la función de aplastamiento

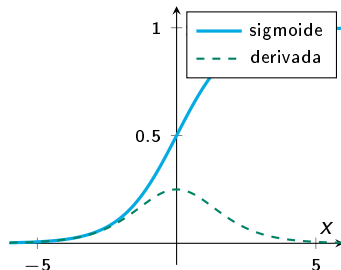
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- ▶ Comprime $\mathbb{R}$ al intervalo $(0, 1)$: útil para interpretar salidas como *probabilidades*.
- ▶ Derivada: $\frac{d}{dx} \text{sigmoid}(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$.

Sigmoide: la función de aplastamiento

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- ▶ Comprime $\mathbb{R}$ al intervalo $(0, 1)$: útil para interpretar salidas como *probabilidades*.
- ▶ Derivada: $\frac{d}{dx} \text{sigmoid}(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$.
- ▶ **Problema:** para $|x|$ grande la derivada $\rightarrow 0 \Rightarrow$ *gradiente se desvanece*.
- ▶ Hoy se usa poco en capas ocultas; sobrevive en compuertas (LSTM/GRU) y en la salida binaria.



Tangente hiperbólica ($\tanh$)

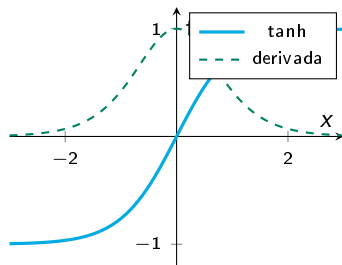
$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- ▶ Comprime $\mathbb{R}$ al intervalo $(-1, 1)$ y es **simétrica** respecto al origen.
- ▶ Derivada:
 $\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$

Tangente hiperbólica ($\tanh$)

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- ▶ Comprime $\mathbb{R}$ al intervalo $(-1, 1)$ y es **simétrica** respecto al origen.
- ▶ Derivada:
 $\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$.
- ▶ Al estar centrada en cero suele converger mejor que la sigmoide.
- ▶ Comparte el problema de saturación para $|x|$ grande.



Propagación hacia adelante (*forward*)

Calcular y almacenar, en orden, las variables intermedias de la red. Para un ejemplo x y una capa oculta (sin sesgos, por brevedad):

$$z = W^{(1)}x \quad (\text{pre-activación})$$

$$h = \sigma(z) \quad (\text{activación oculta})$$

$$o = W^{(2)}h \quad (\text{salida})$$

$$L = \ell(o, y) \quad (\text{pérdida})$$

$$s = \frac{\lambda}{2} (\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2) \quad (\text{regularización } L_2)$$

$$J = L + s \quad (\text{función objetivo})$$

Propagación hacia adelante (*forward*)

Calcular y almacenar, en orden, las variables intermedias de la red. Para un ejemplo x y una capa oculta (sin sesgos, por brevedad):

$$z = W^{(1)}x \quad (\text{pre-activación})$$

$$h = \sigma(z) \quad (\text{activación oculta})$$

$$o = W^{(2)}h \quad (\text{salida})$$

$$L = \ell(o, y) \quad (\text{pérdida})$$

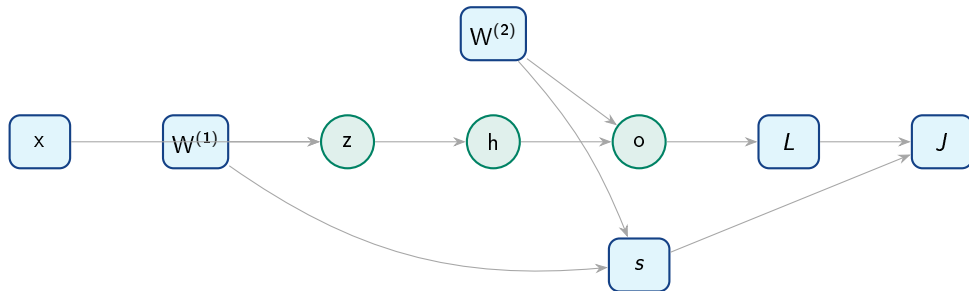
$$s = \frac{\lambda}{2} (\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2) \quad (\text{regularización } L_2)$$

$$J = L + s \quad (\text{función objetivo})$$

Grafo computacional

Representamos el cálculo como un grafo dirigido: los *nodos* son variables y operaciones, las *aristas* las dependencias. Es la base de la *diferenciación automática*.

Grafo computacional



El paso *forward* recorre el grafo de izquierda a derecha. La *retropropagación* lo recorre en sentido inverso, acumulando gradientes con la regla de la cadena.

Retropropagación (*backpropagation*)

Método para calcular el gradiente de J respecto de los parámetros, aplicando la **regla de la cadena** en orden inverso al *forward*.

Retropropagación (*backpropagation*)

Método para calcular el gradiente de J respecto de los parámetros, aplicando la **regla de la cadena** en orden inverso al *forward*.

- Para una composición $z = g(y)$, $y = f(x)$, la regla de la cadena en forma matricial es:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}.$$

Retropropagación (*backpropagation*)

Método para calcular el gradiente de J respecto de los parámetros, aplicando la **regla de la cadena** en orden inverso al *forward*.

- Para una composición $z = g(y)$, $y = f(x)$, la regla de la cadena en forma matricial es:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}.$$

- Se parte del objetivo J y se propaga hacia atrás:

$$\frac{\partial J}{\partial o} = \frac{\partial L}{\partial o},$$

$$\frac{\partial J}{\partial h} = W^{(2)\top} \frac{\partial J}{\partial o},$$

$$\frac{\partial J}{\partial W^{(2)}} = \frac{\partial J}{\partial o} h^\top + \lambda W^{(2)},$$

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial h} \odot \sigma'(z).$$

($\odot$ = producto elemento a elemento).

Entrenar la red: *forward* + *backward*

- 1: Inicializar los parámetros $W^{(1)}, W^{(2)}, \dots$ (ver inicialización).
- 2: **for** cada época **do**
- 3: **for** cada *minibatch* (X, y) **do**
- 4: **Forward**: calcular activaciones y la pérdida J , almacenando las intermedias.
- 5: **Backward**: retropropagar para obtener $\partial J / \partial W^{(l)}$.
- 6: **Actualizar**: $W^{(l)} \leftarrow W^{(l)} - \eta \partial J / \partial W^{(l)}$.
- 7: **end for**
- 8: **end for**

► *Forward* y *backward* se **alternan** y dependen entre sí.

Entrenar la red: *forward* + *backward*

- 1: Inicializar los parámetros $W^{(1)}, W^{(2)}, \dots$ (ver inicialización).
 - 2: **for** cada época **do**
 - 3: **for** cada *minibatch* (X, y) **do**
 - 4: **Forward**: calcular activaciones y la pérdida J , almacenando las intermedias.
 - 5: **Backward**: retropropagar para obtener $\partial J / \partial W^{(l)}$.
 - 6: **Actualizar**: $W^{(l)} \leftarrow W^{(l)} - \eta \partial J / \partial W^{(l)}$.
 - 7: **end for**
 - 8: **end for**
- ▶ *Forward* y *backward* se **alternan** y dependen entre sí.
 - ▶ Hay que **retener** en memoria las variables intermedias del *forward* hasta completar el *backward*: por eso entrenar consume mucha más memoria que predecir.

Gradientes que se desvanecen o explotan

El gradiente en una red de L capas es un **producto** de L matrices jacobianas:

$$\frac{\partial J}{\partial W^{(l)}} \propto \prod_{k=l}^{L-1} \frac{\partial h^{(k+1)}}{\partial h^{(k)}}.$$

- **Explosión:** si los factores son > 1 , el producto crece exponencialmente $\Rightarrow$ parámetros divergen.

Gradientes que se desvanecen o explotan

El gradiente en una red de L capas es un **producto** de L matrices jacobianas:

$$\frac{\partial J}{\partial W^{(l)}} \propto \prod_{k=l}^{L-1} \frac{\partial h^{(k+1)}}{\partial h^{(k)}}.$$

- ▶ **Explosión:** si los factores son > 1 , el producto crece exponencialmente $\Rightarrow$ parámetros divergen.
- ▶ **Desvanecimiento:** si los factores son < 1 , el producto $\rightarrow 0 \Rightarrow$ las capas iniciales dejan de aprender.

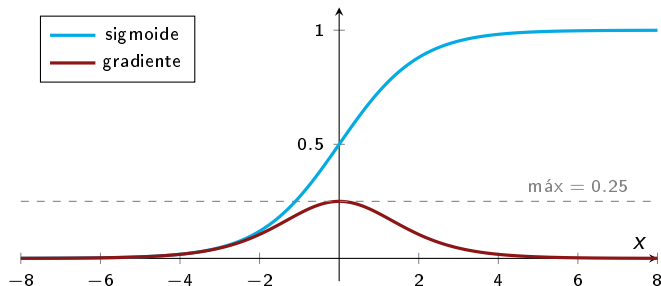
Gradientes que se desvanecen o explotan

El gradiente en una red de L capas es un **producto** de L matrices jacobianas:

$$\frac{\partial J}{\partial W^{(l)}} \propto \prod_{k=l}^{L-1} \frac{\partial h^{(k+1)}}{\partial h^{(k)}}.$$

- ▶ **Explosión:** si los factores son > 1 , el producto crece exponencialmente $\Rightarrow$ parámetros divergen.
- ▶ **Desvanecimiento:** si los factores son < 1 , el producto $\rightarrow 0 \Rightarrow$ las capas iniciales dejan de aprender.
- ▶ La sigmoide es un culpable clásico: su derivada máxima es 0.25 y se anula en las colas (ver siguiente lámina).

El gradiente de la sigmoide se anula en las colas



Cuando la entrada es muy positiva o muy negativa, el gradiente es prácticamente nulo: la retropropagación puede “estancarse”. ReLU evita esto en su rama positiva.

Simetría

Si todos los pesos se inicializan **iguales** (p. ej. a cero), todas las unidades de una capa calculan lo mismo y reciben el mismo gradiente: nunca se diferencian. Hay que inicializar de forma **aleatoria**.

Simetría

Si todos los pesos se inicializan **iguales** (p. ej. a cero), todas las unidades de una capa calculan lo mismo y reciben el mismo gradiente: nunca se diferencian. Hay que inicializar de forma **aleatoria**.

- **Inicialización de Xavier (Glorot)**: elige la varianza de los pesos para mantener estable la magnitud de activaciones y gradientes a través de las capas.

$$\text{Var}(w) = \frac{2}{n_{\text{ent}} + n_{\text{sal}}} \iff w \sim U\left(-\sqrt{\frac{6}{n_{\text{ent}} + n_{\text{sal}}}}, \sqrt{\frac{6}{n_{\text{ent}} + n_{\text{sal}}}}\right).$$

donde n_{ent} y n_{sal} son el número de entradas y salidas de la capa.

Simetría

Si todos los pesos se inicializan **iguales** (p. ej. a cero), todas las unidades de una capa calculan lo mismo y reciben el mismo gradiente: nunca se diferencian. Hay que inicializar de forma **aleatoria**.

- **Inicialización de Xavier (Glorot)**: elige la varianza de los pesos para mantener estable la magnitud de activaciones y gradientes a través de las capas.

$$\text{Var}(w) = \frac{2}{n_{\text{ent}} + n_{\text{sal}}} \iff w \sim U\left(-\sqrt{\frac{6}{n_{\text{ent}} + n_{\text{sal}}}}, \sqrt{\frac{6}{n_{\text{ent}} + n_{\text{sal}}}}\right).$$

donde n_{ent} y n_{sal} son el número de entradas y salidas de la capa.

- Para ReLU se suele usar la variante *He*, con varianza $2/n_{\text{ent}}$.

- ▶ Las redes profundas tienen **enorme capacidad**: pueden memorizar incluso etiquetas aleatorias.

- ▶ Las redes profundas tienen **enorme capacidad**: pueden memorizar incluso etiquetas aleatorias.
- ▶ Por ello la *brecha de generalización* (diferencia entre error de entrenamiento y de prueba) es una preocupación central.

- ▶ Las redes profundas tienen **enorme capacidad**: pueden memorizar incluso etiquetas aleatorias.
- ▶ Por ello la *brecha de generalización* (diferencia entre error de entrenamiento y de prueba) es una preocupación central.
- ▶ Herramientas para mejorar la generalización:
 - ▶ **Detención temprana** (*early stopping*).
 - ▶ **Decaimiento de pesos** (regularización L_2).
 - ▶ **Dropout**.
 - ▶ Más datos y *data augmentation*.

Detención temprana

- ▶ Monitorear el error de validación durante el entrenamiento.
- ▶ Detener cuando el error de validación deja de mejorar, aunque el de entrenamiento siga bajando.
- ▶ Limita de hecho el número de épocas: una forma implícita de regularización.

Decaimiento de pesos (L_2)

- ▶ Añade una penalización a la norma de los pesos:

$$J = L + \frac{\lambda}{2} \sum_l \|W^{(l)}\|_F^2.$$

- ▶ Favorece pesos pequeños $\Rightarrow$ funciones más suaves.
- ▶ λ controla la fuerza de la regularización.

Dropout: la idea

- ▶ Durante el entrenamiento, **desactivar** al azar una fracción p de las neuronas en cada paso.

Dropout: la idea

- ▶ Durante el entrenamiento, **desactivar** al azar una fracción p de las neuronas en cada paso.
- ▶ Impide que las unidades *co-adapten*: ninguna puede depender de la presencia de otra concreta $\Rightarrow$ representaciones más robustas.

Dropout: la idea

- ▶ Durante el entrenamiento, **desactivar** al azar una fracción p de las neuronas en cada paso.
- ▶ Impide que las unidades *co-adapten*: ninguna puede depender de la presencia de otra concreta $\Rightarrow$ representaciones más robustas.
- ▶ Equivale aproximadamente a entrenar y promediar un *ensamble* enorme de subredes que comparten pesos.

Dropout: la idea

- ▶ Durante el entrenamiento, **desactivar** al azar una fracción p de las neuronas en cada paso.
- ▶ Impide que las unidades *co-adapten*: ninguna puede depender de la presencia de otra concreta $\Rightarrow$ representaciones más robustas.
- ▶ Equivale aproximadamente a entrenar y promediar un *ensamble* enorme de subredes que comparten pesos.

Dropout invertido (en la práctica)

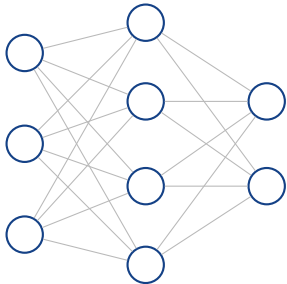
Cada activación h se reemplaza por

$$h' = \begin{cases} 0 & \text{con probabilidad } p, \\ \frac{h}{1-p} & \text{en caso contrario.} \end{cases}$$

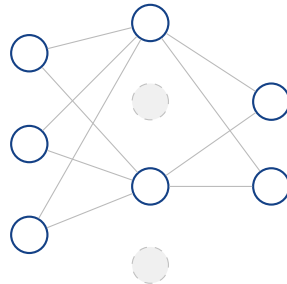
Así $\mathbb{E}[h'] = h$: la salida esperada no cambia y no hay que reescalar al predecir.

Dropout: red completa vs. red con unidades apagadas

Sin dropout



Con dropout



En cada paso de entrenamiento se apaga un subconjunto distinto de neuronas (en gris). Al **predecir**, dropout se desactiva y se usa la red completa.

Caso práctico: precios de viviendas (Kaggle)

Un flujo típico para aplicar un MLP a datos tabulares reales:

1. **Preprocesamiento:** estandarizar variables numéricas (media 0, varianza 1) y codificar las categóricas (*one-hot*).
2. **Métrica:** para precios conviene el *error cuadrático medio en escala logarítmica* (RMSE log), pues importa el error *relativo*.

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (\log \hat{y}_i - \log y_i)^2}$$

3. **Validación cruzada K-fold:** estimar el error de generalización y ajustar hiperparámetros (capas, ancho, λ , *learning rate*).
4. **Entrenar** con todos los datos y generar predicciones.

Concepto	Idea clave
MLP	Capas totalmente conectadas + no linealidad
Activación	ReLU por defecto; sigmoide/tanh saturan
No linealidad	Sin ella, la red colapsa a un modelo lineal
Backprop	Regla de la cadena en sentido inverso
Estabilidad	Gradientes que se desvanecen / explotan
Inicialización	Romper simetría; Xavier/He
Regularización	<i>Early stopping</i> , L_2 , dropout

- ▶ Un MLP es la red profunda más simple, pero ya contiene **todas** las ideas centrales: capas, no linealidad, retropropagación y regularización.
- ▶ Estas bases se reutilizan en CNN, RNN y *transformers*.

- ▶ Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 5 (Multilayer Perceptrons)
https://d2l.ai/chapter_multilayer-perceptrons/index.html
- ▶ Goodfellow, Bengio & Courville, *Deep Learning* (cap. 6, *Deep Feedforward Networks*)
<https://www.deeplearningbook.org/>
- ▶ Glorot & Bengio (2010), *Understanding the difficulty of training deep feedforward neural networks*.
- ▶ Srivastava et al. (2014), *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*.
- ▶ Documentación de PyTorch: *torch.nn*
<https://pytorch.org/docs/stable/nn.html>